



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ЛИПЕЦКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»**

Институт (факультет) Компьютерных наук

Кафедра Прикладной математики и системного анализа

КУРСОВАЯ РАБОТА

По дисциплине: «Современные платформы разработки ПО».

На тему: «Реализация эмулятора Машины Тьюринга».

Студент

ПМ-24-1

группа

подпись, дата

Сараев А.П.

фамилия, инициалы

Руководитель

К.Т.Н.

ученая степень, ученое звание

подпись, дата

Немец С.Ю.

фамилия, инициалы

Липецк 2026

Аннотация

В курсовой работе приводится реализация эмулятора Машины Тьюринга. Каждый пользователь имеет возможность создать Машину Тьюринга со своими параметрами и просимулировать её работу. Полный код приложения: https://github.com/Rozb0nd/LSTU/tree/main/sem_4/SPRP0/Kurosovaya.

Задание кафедры

Программный продукт должен иметь интерфейс пользователя с эргономичным расположением управляющих элементов и помимо функций решения задачи темы курсовой работы должен включать в себя:

1. Ведение журнала действий пользователей и изменения текущего состояния программы.
2. Инструкцию по запуску программы в отдельном тестовом файле.
3. Возможности регистрации и авторизации пользователей.
4. Генерация отчётов.
5. Обработчик ошибок как пользователя (выдача рекомендаций при неверных действиях), так и выполнения (предупреждение при возникновении ошибки в работе программы).

Оглавление

1 Основная часть	4
1.1 Описание программы	4
1.1.1 Теория модели	4
1.1.2 Функции программы	4
1.1.3 Архитектура	6
1.2 Текст программы	6
1.3 Примеры работы программы	11
1.4 Вывод	12
Список литературы	12

1 Основная часть

1.1 Описание программы

1.1.1 Теория модели

Программа представляет собой интерактивный эмулятор машины Тьюринга — классической абстрактной модели вычислений, предложенной Аланом Тьюрингом в 1936 году.

Машина Тьюринга (МТ) состоит из двух частей — ленты и автомата. Лента используется для хранения информации. Она бесконечна в обе стороны и разбита на клетки, которые никак не нумеруются и не именуется. Автомат — это активная часть МТ. В каждый момент он размещается под одной из клеток ленты и видит её содержимое; это видимая клетка, а находящийся в ней символ — видимый символ; содержимое же соседних и других клеток автомат не видит [1].

1.1.2 Функции программы

При запуске программы пользователь попадает в диалог авторизации, где он может войти под своей учётной записью или зарегистрировать новую.

После входа открывается главное окно, разделённое на несколько функциональных зон. В верхней части расположена панель настроек, где пользователь задаёт базовые параметры будущей машины Тьюринга. Он указывает алфавит — набор символов, с которыми будет работать машина (символы вводятся подряд, без пробелов). Отдельно задаётся пустой символ, который обозначает пустую ячейку на бесконечной ленте; этот символ обязан присутствовать в алфавите. Далее пользователь вводит содержимое начальной ленты, задаёт начальное состояние машины и выбирает режим останова. Режимов останова два: либо одно конечное состояние q_t (классический останов), либо пара состояний q_u и q_p , соответствующих допуску и отвержению входной строки. При этом конечные состояния добавляются в систему автоматически и не требуют ручного ввода.

Центральное место в интерфейсе занимает таблица переходов. Каждая строка таблицы соответствует одному состоянию машины, а каждый столбец — одному символу, который может быть прочитан с ленты. На пересечении

строки и столбца пользователь записывает правило перехода в формате «следующее состояние, записываемый символ, направление движения головки». Направление обозначается буквами R (вправо), L (влево) или N (остаться на месте). Пользователь может свободно добавлять и удалять состояния и символы. Имена состояний qt, qu и qp зарезервированы и не могут быть добавлены вручную — они появятся автоматически при создании машины.

После заполнения всех полей пользователь нажимает кнопку «Применить настройки и сбросить». На этом этапе программа выполняет полную валидацию введённых данных. Проверяется, что алфавит не пуст и не содержит повторяющихся символов, что пустой символ — это ровно один символ и он есть в алфавите, что начальная лента состоит только из символов алфавита, что начальное состояние не пусто. Затем анализируется таблица переходов: каждая ячейка разбирается на три компонента, проверяется существование указанного следующего состояния среди состояний таблицы или среди конечных состояний, проверяется, что записываемый символ присутствует в алфавите, а направление указано корректно.

В нижней части окна отображается лента — последовательность ячеек, каждая из которых содержит один символ. Текущая позиция головки подсвечивается красной рамкой и стрелкой. При движении головки панель ленты автоматически прокручивается, удерживая головку в видимой области. Рядом с лентой выводится текущее состояние машины и, в случае останова, пометка «ОСТАНОВЛЕНА».

Управление выполнением осуществляется кнопками в нижней части окна. Кнопка «Шаг вперёд» выполняет ровно один переход в соответствии с таблицей: читает символ под головкой, находит правило для текущего состояния и прочитанного символа, записывает новый символ на ленту, сдвигает головку в указанном направлении и переходит в следующее состояние. Каждый шаг сохраняется в истории, что позволяет в любой момент нажать «Шаг назад» и откатиться к предыдущему состоянию. Откат возможен на любое количество шагов в пределах текущей сессии. Кнопка «Выполнить до останова» запускает автоматический режим, при котором шаги выполняются с интервалом в 200 миллисекунд до тех пор, пока машина не достигнет одного из конечных состояний или не встретит отсутствие правила для текущей ситуации (что также приводит к останову). В любой момент выполнение можно

остановить кнопкой «Стоп».

1.1.3 Архитектура

Основные классы модели, такие как `TuringMachine`, `Tape`, `Snapshot`, `TransitionTable`, `Transition` и `TransitionKey`, инкапсулируют всю логику, связанную с работой абстрактной машины Тьюринга, и не зависят от пользовательского интерфейса. Лента реализована на основе двух двусторонних очередей, что позволяет эффективно выполнять операции чтения, записи и сдвига головки. Таблица переходов хранится как отображение, ключом которого выступает пара «состояние и текущий символ», а значением — объект перехода, содержащий следующее состояние, записываемый символ и направление. Снимки состояний машины позволяют реализовать пошаговый откат без повторного выполнения с начала.

Графический интерфейс построен на библиотеке `Swing` и отделён от модели. Класс `TuringMachineApp` создаёт все визуальные компоненты, обрабатывает действия пользователя, выполняет валидацию ввода и синхронизирует модель с отображением. Для работы с файлами выделен класс `FileManager`, использующий библиотеку `Jackson` для сериализации и десериализации `JSON`. Управление пользователями вынесено в `UserManager`, а управление коллекцией сохранённых машин — в `TMManager`. Отдельный класс `ReportGenerator` отвечает за формирование PDF-отчётов.

Из сторонних библиотек используются `Lombok` для сокращения шаблонного кода, `Jackson` для работы с `JSON`, `BCrypt` для хеширования паролей, `iText` для генерации PDF и `SLF4J` для логирования.

1.2 Текст программы

Листинг 1 – Реализация Машины Тьюринга

```
1 package TuringMachine;
2
3 import TuringMachine.Table.Moves;
4 import TuringMachine.Table.Transition;
5 import TuringMachine.Table.TransitionTable;
6 import com.fasterxml.jackson.annotation.JsonIgnore;
7 import lombok.Getter;
8 import lombok.Setter;
9
10 import java.util.ArrayList;
```

```

11 import java.util.List;
12 import java.util.Set;
13
14 @Getter
15 @Setter
16 public class TuringMachine {
17     private Set<Character> alphabet;
18     private Tape tape;
19     private TransitionTable table;
20     private boolean halted = false;
21     private String currentState;
22     private Set<String> haltStates;
23     private List<Snapshot> history = new ArrayList<>();
24     private String owner;
25     private String name = "";
26     private String initialTape;
27
28     public TuringMachine(Set<Character> alphabet, String tape, Character blank,
29         String currentState, Set<String> haltStates, String owner) {
30         this.alphabet = alphabet;
31         this.tape = new Tape(tape, blank);
32         this.currentState = currentState;
33         this.haltStates = haltStates;
34         this.table = new TransitionTable();
35         this.owner = owner;
36         this.initialTape = tape;
37     }
38
39     public TuringMachine() {};
40
41     public TuringMachine(TuringMachine other) {
42         this.alphabet = other.alphabet;
43         this.tape = other.tape;
44         this.currentState = other.currentState;
45         this.haltStates = other.haltStates;
46         this.table = other.table;
47         this.owner = other.owner;
48         this.name = other.name;
49         this.halted = other.halted;
50         this.history = other.history;
51         this.initialTape = other.initialTape;
52     }
53
54     public void setTransitionTable(TransitionTable table) {
55         this.table = table;
56     }
57
58     public void addTransition(String state, Character currentChar,

```

```

59         String nextState, Character writeChar,
60         Moves move) {
61     table.addTransition(state, currentChar,
62         nextState, writeChar, move);
63 }
64
65 public void removeTransition(String state, Character currentChar) {
66     table.removeTransition(state, currentChar);
67 }
68
69 public void nextMove() {
70     history.add(new Snapshot(tape, currentState));
71     if (halted) return;
72
73     Character currentChar = tape.read();
74     Transition transition = table.getTransition(currentState, currentChar);
75     if (transition == null) {
76         halted = true;
77         return;
78     }
79     tape.write(transition.getWriteChar());
80     Moves move = transition.getMove();
81     switch (move) {
82         case LEFT: tape.leftMove(); break;
83         case RIGHT: tape.rightMove(); break;
84         case STAY: break;
85     }
86     currentState = transition.getNextState();
87     if (haltStates.contains(currentState)) {
88         halted = true;
89     }
90 }
91
92 public void previousMove() {
93     if (history.isEmpty()) return;
94     Snapshot previous = history.remove(history.size() - 1);
95     this.tape = previous.tape;
96     this.currentState = previous.state;
97     halted = false;
98 }
99
100 public void runToEnd() {
101     while (!halted) nextMove();
102 }
103
104 @JsonIgnore
105 public boolean isHalt() {
106     return halted;

```



```

107     }
108
109     @JsonIgnore
110     public List<Character> getFullTape() {
111         return tape.getFullTape();
112     }
113
114     public Tape TapeClassReturn() { return tape;}
115
116     @JsonIgnore
117     public String getState() {
118         return currentState;
119     }
120
121     @JsonIgnore
122     public int getHeadAbsolutePosition() {
123         return tape.getHeadPos();
124     }
125
126     public List<Snapshot> getHistory() {
127         return history;
128     }
129
130     public String getInitialTape() {
131         return initialTape;
132     }
133
134     public void clearHistory() {history.clear();}
135 }

```

Листинг 2 – Реализация ленты

```

1 package TuringMachine;
2
3 import com.fasterxml.jackson.annotation.JsonIgnore;
4 import lombok.Getter;
5
6 import java.util.ArrayDeque;
7 import java.util.ArrayList;
8 import java.util.Deque;
9 import java.util.List;
10
11 @Getter
12 public class Tape {
13     private Deque<Character> leftTape, rightTape;
14     private char blank;
15     private int headPos;

```

```

16
17 public Tape(String start, char blank) {
18     leftTape = new ArrayDeque<>();
19     rightTape = new ArrayDeque<>();
20     headPos = 0;
21     for (Character c : start.toCharArray()) {
22         rightTape.addLast(c);
23     }
24     if (rightTape.isEmpty()) rightTape.addLast(blank);
25     this.blank = blank;
26 }
27
28 public Tape() {}
29
30 public Tape(Tape other) {
31     this.leftTape = new ArrayDeque<>(other.leftTape);
32     this.rightTape = new ArrayDeque<>(other.rightTape);
33     this.blank = other.blank;
34     this.headPos = other.headPos;
35 }
36
37 public Character read() {
38     return (!rightTape.isEmpty() ? rightTape.getFirst() : blank);
39 }
40
41 public void write(Character c) {
42     if (rightTape.isEmpty()) rightTape.addFirst(blank);
43     rightTape.removeFirst();
44     rightTape.addFirst(c);
45 }
46
47 public void rightMove() {
48     if (rightTape.isEmpty()) rightTape.addFirst(blank);
49     else leftTape.addLast(rightTape.removeFirst());
50     headPos++;
51 }
52
53 public void leftMove() {
54     if (leftTape.isEmpty()) rightTape.addFirst(blank);
55     else rightTape.addFirst(leftTape.removeLast());
56     headPos--;
57 }
58
59 @JsonIgnore
60 public List<Character> getFullTape() {
61     List<Character> fullTape = new ArrayList<>();
62     fullTape.addAll(leftTape);
63     fullTape.addAll(rightTape);

```

```

64     return fullTape;
65 }
66
67 @Override
68 public String toString() {
69     return getFullTape().toString();
70 }
71
72 public int getHeadPos() {
73     return leftTape.size();
74 }
75 }

```

1.3 Примеры работы программы

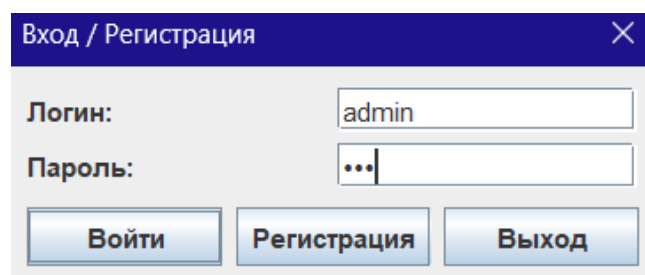


Рис. 1.1 – Окно авторизации/регистрации

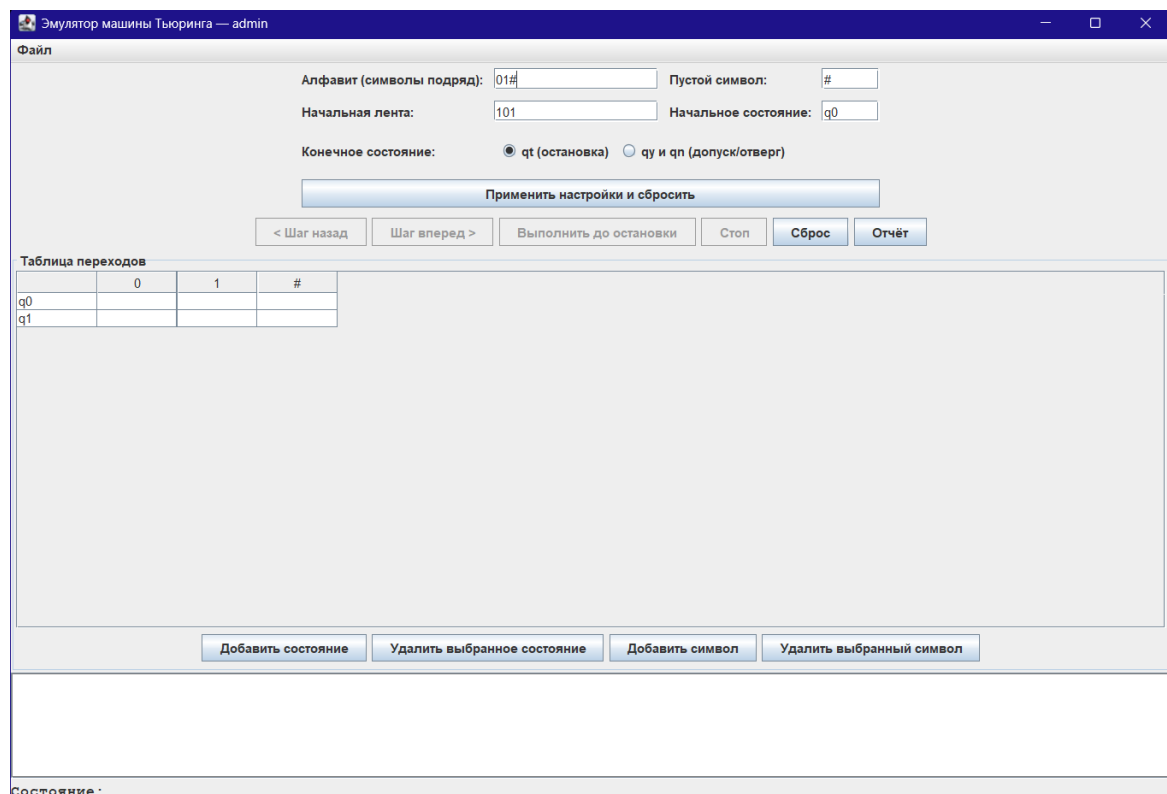


Рис. 1.2 – Стартовое меню

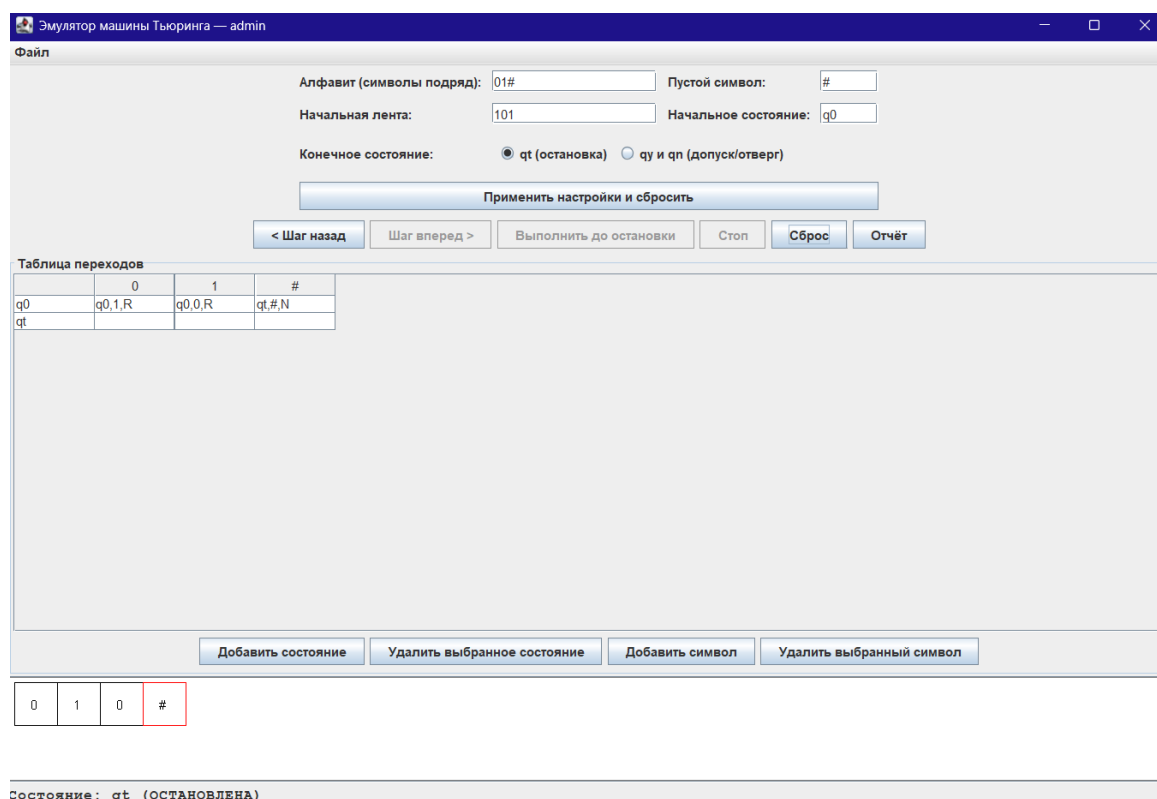


Рис. 1.3 – Выполнение работы Машины Тьюринга

1.4 Вывод

В ходе выполнения курсовой работы был разработан эмулятор машины Тьюринга с графическим интерфейсом на языке Java. Приложение поддерживает:

- настройку алфавита, начальной ленты и системы переходов через интерактивную таблицу;
- два режима остановки: одиночное состояние qt или пара qu/qn ;
- пошаговое и автоматическое выполнение программы с визуализацией ленты;
- сохранение и загрузку конфигураций машин.

Разработанное программное средство может применяться в учебном процессе при изучении теории алгоритмов и формальных моделей вычислений.

Список литературы

1. дp B. П. и. Машина Тьюринга и алгоритмы Маркова. Решение задач. — МГУ им. М.В. Ломоносова, 2016.